

2.3 MCP: Model Context Protocol v2

2.3 MCP: Model Context Protocol——把外部能力接入 Agent

大家好，上一节我们完成了 Tool Runtime。

现在，无论模型提出什么 Tool Call，都要先经过统一的工具定义、参数校验和执行入口，再把 Tool Result 交回 Agent Loop。

但是上一节的工具仍然主要注册在当前进程里。

真实项目中的订单服务、数据库、GitHub、文件系统和企业内部 API，不会都和 Agent 写在同一个代码仓库，也不会使用同一种 SDK。

这就产生了本节要解决的问题：**Tool Runtime 已经会执行工具了，我们怎样把不同团队、不同进程、不同技术栈提供的能力，用同一种方式接进来？**

先给结论：**MCP 是 Agent Host 与外部能力之间的标准接入协议。它负责发布、发现和调用能力；它不替代 Function Calling，不替代上一节的 Tool Runtime，也不替代 Agent Loop。**

我们继续使用订单查询 Agent。上一节的 `get_order` 是本地函数；这一节把它移动到独立的 MCP Server，再由 Host 中的 MCP Client 发现并调用。整个改造只替换工具的接入方式，Runtime 的校验入口和 Loop 的执行方式不重写。

这正是 Agent 开发工程师需要掌握的能力。

AI 可以很快生成 `@mcp.tool()`、Client 连接和类型注解，但“代码能运行”不是最终价值。

工程师真正要交付的是：外部能力能够被稳定接入，工具契约不会漂移，失败能够被区分，远端调用不会绕过 Runtime，结果能够回到 Loop 继续完成用户任务。

所以，本节的学习重点不是背协议，我们要学习怎样驾驭 AI，把 MCP 代码变成可以控制、验证和迁移的工程能力。

一、为什么要学 MCP

1.1 学完以后有什么用？

学完以后，你应该能够接手这样一项真实工作：**团队已经有订单、数据库或代码仓库能力，你要把它封装成 MCP Server，让现有 Agent 能发现、调用并继续执行任务，同时不破坏原来的 Tool Runtime 和 Agent Loop。**

具体来说，你要形成四种能力：

- 1. **能解释**：说清 Function Calling、MCP、Tool Runtime 和 Agent Loop 的分工；
- 2. **能构建**：把一个内部能力封装成 MCP Server，并在 Host 中完成发现和调用；
- 3. **能验证**：用测试证明参数拒绝发生在远端执行之前，业务失败不会被误判成网络故障，结果能够关联回原 Tool Call；
- 4. **能迁移**：把订单案例迁移到文件系统、数据库、GitHub 或内部 HTTP 服务，而不是重新发明一套接入协议。

我需要你学完以后，能讲清楚工具为什么这样拆、MCP Adapter 为什么放在 Runtime 的执行边界、失败后 Loop 为什么继续或停止，这才叫掌握 MCP 的 Agent 工程能力。

1.2 哪些必须掌握，哪些只需要了解？

学习层级	内容	本节验收标准
必须掌握	Host、Client、Server、Tool、Resource、Prompt、Transport	能画出位置并解释谁调用谁
必须掌握	Server 发布能力，Client 发现并调用能力	能跑通 stdio 端到端链路
必须掌握	MCP Tool 转换成 Runtime 的 <code>ToolDefinition</code>	能接回现有 Runtime 与 Loop
必须掌握	限定名、结果归一化、调用关联、失败分类	能验证成功、拒绝、失败和恢复路径
理解即可	Streamable HTTP、协议探测、旧版本兼容	能做技术选择，不手写协议编解码

了解后续	OAuth、权限、审批、审计、多租户、可观测性	知道插入位置，本节不展开平台实现
------	-------------------------	------------------

JSON-RPC 的所有字段、每个历史协议版本、SDK 内部编解码器都不要求背。旧 HTTP + SSE、旧 `initialize` 握手和旧版导入路径，看到时能够识别即可。**新项目锁定当前 SDK，让 Adapter 吸收协议变化，不让业务 Tool 到处判断版本。**

接下来我们先把术语和主要流程统一。只有知道 MCP 放在哪里，后面的代码才不会把职责放错层。

二、先统一术语：MCP 到底接在什么位置？

2.1 为什么必须先统一术语？

先给结果：在 MCP 架构中，Host 就是运行 Agent 和 MCP Client 的进程环境（即上一节我们构建的 Agent Host），它负责发起工具调用、管理连接生命周期，并统筹整个 Agent 循环。

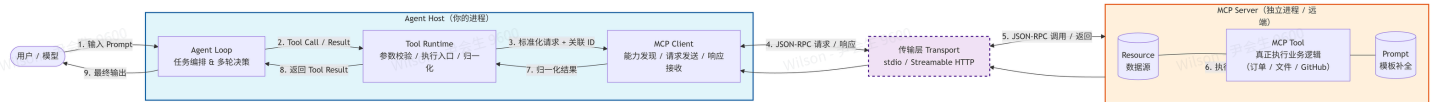
Client 是 Host 内部与 MCP Server 通信的协议组件，它负责发现远端能力、发送请求（如初始化、工具调用、资源读取）并接收响应，是 Host 与外界能力的唯一交互出口。

MCP Tool 是由 MCP Server 发布的可调用函数——它只在 Server 端实际执行，Client 通过协议发起调用请求并等待结果，本地并不执行任何业务逻辑。

这三个角色的调用关系是：Host → Client → Server → Tool。Host 不直接调用远端函数，而是通过 Client 将请求转发给 Server，Server 执行后返回结果，再由 Client 交回 Host 的 Tool Runtime，最终进入 Agent Loop。

如果这些术语被混淆，职责就容易错层——比如把远端能力误塞进模型推理层，或者把连接管理混入业务工具逻辑中。

因此，在动手编码之前，我们必须先对齐这些核心概念的位置与调用关系。只有术语统一，后面的架构拆解、异常分类和迁移方案才能站得住脚。



术语	本节统一含义	不负责什么
Function Calling	模型输出工具名与结构化参数的机制	不负责真实执行和授权
MCP	Host 与外部 Server 交换工具、资源和 Prompt 的标准协议	不提供完整 Agent Loop 和业务治理

Host	承载 Agent、用户会话和整体编排的应用	不等于某个 MCP Server
MCP Client	Host 内连接一个 MCP Server 的协议客户端	不直接替代 Tool Runtime
MCP Server	按协议暴露能力的一端	不代表该能力天然可信
Tool	可调用、可能产生动作的能力	不是纯上下文的唯一表达方式
Resource	用 URI 标识的只读上下文	不应该伪装成写操作
Prompt	Server 提供的参数化任务模板	不承担确定性业务约束
Transport	消息如何在两端传输，本节使用 stdio 与 Streamable HTTP	不决定工具是否允许执行
Tool Runtime	Host 中所有工具调用共同经过的执行入口	不负责模型下一轮决策
Agent Loop	模型、Tool Call、Tool Result 与终止条件之间的循环	不应直接依赖每个远端 API 的细节

这里最容易混淆的是 Function Calling 和 MCP。

模型看到 `get_order` 的描述，决定输出：

代码块

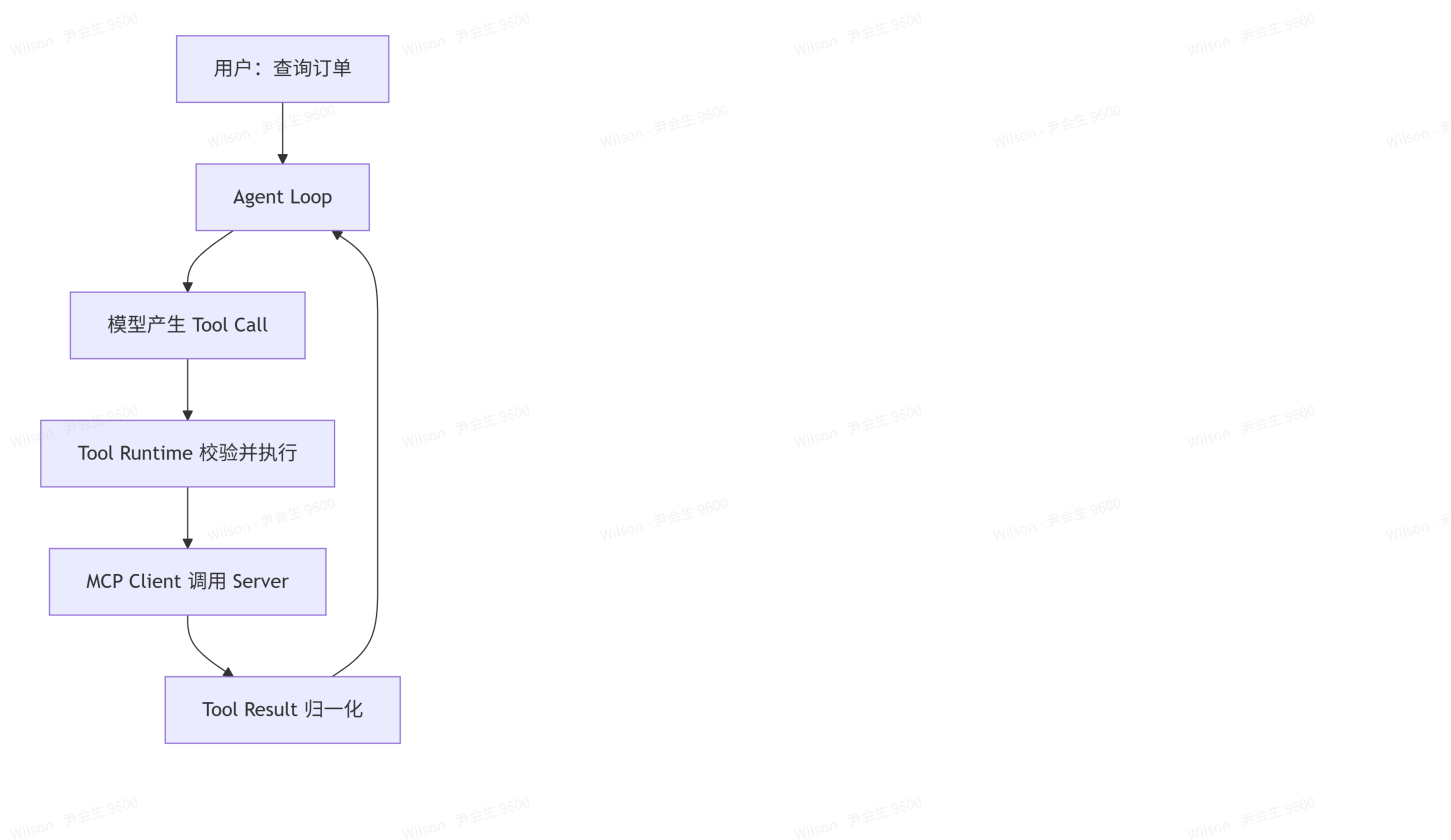
```
1  {
2    "name": "orders.get_order",
3    "arguments": {"order_id": "ord_1001"}
4  }
```

这是 Function Calling。Host 收到候选动作以后，通过 MCP Client 把请求送到订单 Server，这是 MCP。

调用之前由 Runtime 校验参数，调用之后由 Loop 把结果写回消息，再决定是否继续。这四件事不能互相替代。

2.2 本节的主要路径是什么？

我们先看全局，再进入细节：



在模型发起调用之前，还有一条启动链：

代码块

- 1 连接 Server
- 2 → 发现 Tools / Resources / Prompts
- 3 → 给工具加 server_id 前缀
- 4 → 转换为 Runtime 的 ToolDefinition
- 5 → 生成模型可见的 Function Calling Schema
- 6 → 保存为本次 Run 的工具快照

为什么必须走这条启动链？

上一节的 Runtime 已经知道怎样执行一个 `ToolDefinition` ——校验参数、构造调用、归一化结果。MCP 只需要把远端工具适配成同一种内部接口 `ToolDefinition`，Runtime 和 Loop 的其余逻辑一个字都不需要改。也就是说，**工具是本地函数还是远端 MCP Server，对 Loop 完全透明。**

对 Agent 开发工程师而言，本节最值得迁移的设计思想是：**协议对象只停留在 Adapter 内部，Runtime 和 Loop 只面向稳定的内部对象。**以后切换 MCP SDK 升级版本、从 stdio 换成 Streamable HTTP，改的是连接与适配层，而不是整条 Agent 主链。这个边界如果守住了，MCP 版本更新对你来说只是一个 Adapter 的小版本升级，而不是整个 Agent 系统的重写。

2.3 这些知识点为什么不能平均用力？

从生产级 Agent 的实际使用强度来看，本节不是一份 MCP 名词百科。真正高频的只有三件事：**Server 如何发布能力、Host 如何发现并转换工具、Loop 如何回灌结果**。Resources 与 Prompts 是需要认识的协议原语，但并非订单案例的运行主轴；权限审计和多租户虽重要，也应在真实调用链跑通之后再设计。

可以用四个问题判断学习优先级：

观察位置	本节高频问题	本节处理方式
MCP SDK	怎样发布、发现和调用能力	详细讲并运行代码
Agent Harness	怎样聚合多个 Server，并形成稳定内部 Tool	详细讲 Adapter 与限定名
Agent Loop	Tool Result 怎样关联、回灌并触发下一轮	详细讲成功、失败与停止
生产环境	谁能调用、怎样审计、连接怎样隔离	只确定插入位置，后续展开

这也是为什么我们不会用四个互不相关的案例分别讲文件系统、数据库、GitHub 和内部 HTTP——无论底层能力是什么，**Host 主链必须保持不变**。先用订单案例把主链跑通，后续只更换 Server 内部实现和业务策略，才能验证抽象是否真的成立。

本阶段解决了“术语如何对应、MCP 放在哪里”的问题。你需要形成的能力是**画清职责边界**——知道什么该放在 Adapter 里，什么绝不能越过 Runtime 进入 Loop。但位置清楚之后，下一个问题自然浮现：**两端究竟怎样确认协议版本并交换消息？**

三、第一阶段——协议地基：Client 与 Server 怎样说同一种语言？

3.1 需要读懂 JSON-RPC

MCP 消息建立在 JSON-RPC 2.0 之上。你至少要能区分请求、响应、通知和错误，才能判断故障发生在模型、Runtime、传输层还是 Server。

一个工具调用在概念上接近下面这组消息：

代码块

```
1  {
2    "jsonrpc": "2.0",
3    "id": 17,
4    "method": "tools/call",
```

```
5  "params": {
6    "name": "get_order",
7    "arguments": {"order_id": "ord_1001"}
8  }
9 }
```

代码块

```
1  {
2    "jsonrpc": "2.0",
3    "id": 17,
4    "result": {
5      "content": [{"type": "text", "text": "订单已发货"}],
6      "structuredContent": {
7        "order_id": "ord_1001",
8        "status": "shipped"
9      }
10 }
11 }
```

其中：`id` 用于将响应关联回请求；`method` 表示协议动作；`params` 是输入；`result` 和 `error` 是互斥的两条结果路径。通知与请求的区别在于没有 `id`，接收方不会返回对应响应。

工程师不需要背诵消息全文，但必须守住两个检查点：相关性——并发调用时不能把 A 的响应交给 B；失败语义——JSON-RPC 错误、MCP Tool Error、业务返回 `ok=false` 是三种不同性质的失败，Host 侧不能全部压成一个字符串扔给 Loop。

为了定位问题，可以先把失败分成四层：

失败层	订单案例	谁先处理
Transport	子进程退出、HTTP 断连、请求超时	Client / Connection Manager
Protocol	方法不存在、版本不兼容、响应关联错误	MCP SDK 与 Adapter
Tool	<code>is_error=true</code> 、 <code>ORDER_NOT_FOUND</code>	Runtime 归一化后交给 Loop
Business	订单存在但当前状态不能执行后续动作	业务 Tool 与 Loop 重新规划

如果把这四层全部变成 `Exception("tool failed")`，后续系统就无法判断该重连、该修参数、该换计划还是该停止。AI 可以帮你补异常捕获，但错误分类必须由工程师先定义。错误结构一旦

被 Loop、日志和测试依赖，它就已经是 Harness 协议的一部分。

3.2 当前规范还必须先走 `initialize` 吗？

这里需要修正很多旧教程留下的印象。

当前规范已将协议元数据放入请求中，不再强制要求旧式 `initialize` → `notifications/initialized` 作为所有新连接的起点；`server/discover` 用于获取 Server 身份、能力和版本信息，同时支持兼容探测。

官方 Python SDK 的 Client 会优先探测现代 Server，必要时自动回退到旧版 `initialize` 流程。

因此，本节的学习方式不是手绘并死记一条过时握手，而是理解三个稳定问题：

1. 我连接的是谁；
2. 双方能使用哪些协议能力；
3. 版本不兼容时，是明确失败还是进入兼容路径。

路线图中的 `initialize`、`capabilities` 和 `notifications/initialized` 仍需认识——你会遇到旧 Server、旧日志和旧教程。但新代码应让 SDK 处理协商，不要在业务层自行拼协议包。

版本策略结论前置：新项目锁定并测试一个 SDK 版本，协议兼容交给 Adapter，不要让业务 Tool 到处判断版本号。

升级时至少回归：发现结果、输入 Schema、结构化输出、Tool Error 和两种 Transport。版本号不需要背，但依赖必须锁定——否则同一份课堂代码在不同日期安装，可能得到不同 API 行为。

3.3 Tools、Resources、Prompts 应该怎样区分？

先给最实用的判断依据：

- 有动作的能力 → Tool（需要参数并执行查询或副作用操作）
- 只读且有稳定 URI 的上下文 → Resource（可重复读取，不产生副作用）
- 需要用户显式选用的任务模板 → Prompt（可复用入口，非业务规则）

在订单案例中：

原语	示例	为什么这样放
Tool	<code>get_order(order_id)</code> 、 <code>create_ticket(order_id, reason)</code>	需要参数并执行查询或动作
Resource	<code>policy://order-status</code>	是可重复读取的状态说明，不产生副作用
Prompt	<code>order_assistant(order_id)</code>	是可复用任务入口，不是真实业务规则

Tools 是本节重点。 Resources 只讲只读定位；Prompts 讲到发现和调用即可。不要为了“完整使用 MCP”而把同一份能力复制成三种原语。

原语选择属于**工程设计**，模型只能根据你给出的业务事实辅助生成声明，不能替你决定职责边界。

本阶段解决了“协议消息与三类能力怎样理解”的问题。你需要形成的能力是：**读懂契约、识别版本、选择原语**。协议地基已经打好，下一步才是把真实订单系统封装成一个可被发现的 MCP Server。

四、第二阶段——发布能力：怎样写一个边界清楚的 MCP Server?

4.1 Server 最重要的工作是什么?

结论前置：**Server 最重要的不是装饰器写法，而是把内部系统收敛成稳定、任务级、可验证的能力边界。**

不要把订单服务的全部 HTTP 路由原样暴露给模型。内部接口可能允许任意更新 `status` 字段，但 Agent 真正需要的是 `create_ticket`。任务级 Tool 缩小了参数空间，也让描述、错误码和后续权限策略有明确落点。

下面是 Server 的关键片段：

mcp_server_v1.py

代码块

```
1  from mcp.server import MCPServer
2  from mcp.server.mcpserver.exceptions import ToolError
3
4  mcp = MCPServer(
5      "order-service",
6      instructions="查询订单必须先调用 get_order，不要猜测订单状态。",
7  )
8
```

```
9 @mcp.tool(structured_output=True)
10 def get_order(order_id: str) -> dict[str, object]:
11     order = ORDERS.get(order_id)
12     if not order:
13         raise ToolError("ORDER_NOT_FOUND")
14     return {"ok": True, **order}
```

这段代码解决的是“怎样把内部查询能力变成可发现的 MCP Tool”。三个最关键的设计点：

- **类型注解**用于生成输入 Schema，Host 不必再手写一份容易漂移的声明；
- `structured_output=True` 让 Host 优先拿到结构化结果，无需从自然语言中解析订单状态；
- `ORDER_NOT_FOUND` 是稳定的失败语义，模型可据此换订单号，Runtime 不应将其误判为网络故障并盲目重试。

AI 可以根据已确定的接口生成这段包装代码，但工程师必须亲自决定：订单号格式、真实数据源、错误码、以及哪些字段允许返回。迁移到数据库或 GitHub 时，迁移的是“任务级工具 + 结构化结果 + 稳定错误”的设计模式，而不是复制 `ORDERS` 这个字典。

设计一个 Server Tool 时，建议工程师逐项回答六个问题：

问题	检查要点
Tool 名称表达的是业务任务，还是泄漏了内部 HTTP 路由？	名称应是意图驱动，非实现驱动
描述是否说明何时使用、何时不能使用？	不只是改写函数名
哪些参数可由模型提出，哪些必须由 Host 注入？	身份与租户信息绝不来自模型
返回给模型的是完成下一步所需字段，还是整张数据库记录？	最小必要信息原则
失败是否有稳定代码，调用方能否区分不可重试与可恢复？	失败分类是契约的一部分
该动作是否幂等，重复到达会不会创建两张工单？	幂等键由 Server 负责，不依赖模型

以 `tenant_id` 为例：**它不应让模型自由填写**。模型可以提出 `order_id` 和售后原因，但当前用户、租户和授权凭证必须来自可信会话。如果把它们都写进 Tool Schema，模型就可能“选择”另一个租户。这不是 Prompt 写得更严格就能解决的问题，而是**契约放错了数据来源**。

`create_ticket` 也不能因为示例返回固定编号，就被理解为生产写法。真实实现要把幂等键、业务事务和下游请求 ID 交给订单系统处理。**MCP Server 是适配边界，不应该把原有业务一致性责任转移给模型。**

Resource 与 Prompt 的代码边界

只需要看到它们与 Tool 的职责差异：

代码块

```
1 @mcp.resource("policy://order-status")
2 def order_status_policy() -> str:
3     return "pending=待处理; shipped=已发货; cancelled=已取消"
4
5 @mcp.prompt()
6 def order_assistant(order_id: str) -> str:
7     return f"查询订单 {order_id}。必须使用工具获取真实状态，不要猜测。"
```

- Resource 提供只读说明，无副作用；
- Prompt 提供任务模板，不替代业务规则。

真正的订单状态仍然来自 `get_order`，不能由 Prompt 猜测；“只允许查询本人订单”也不能只写在 Prompt 里——那是后续治理与业务鉴权必须执行的确定性规则，不能委托给模型自觉。

4.2 stdio 与 Streamable HTTP 怎么选？

本节只学习两种当前主线传输：

传输	适合场景	工程师需要决定什么
stdio	Host 启动本地子进程，本地文件或开发工具	可执行文件、参数、工作目录、环境变量、子进程退出策略
Streamable HTTP	团队共享、独立部署的远程 Server	地址、鉴权、超时、网络边界、部署与容量

Server 启动代码很短：

代码块

```
1 if "--http" in sys.argv:
2     mcp.run("streamable-http", host="127.0.0.1", port=8000)
3 else:
4     mcp.run("stdio")
```

这段代码只是在选择传输，不是在完成连接治理。stdio 子进程是否自动重启、远端 Token 怎样获取、不同租户能否共享连接——这些仍由 Host 的连接管理器设计，不在 Server 的启动代码里解决。旧 HTTP + SSE 只需在维护历史系统时识别，新项目不把它作为实现目标。

选择传输时也不要只看“本地还是远程”：

- stdio 的信任边界包含本地可执行文件、启动参数、工作目录和环境变量。Host 如果把全部进程环境原样转发，可能把不必要的密钥交给子进程；如果允许模型控制启动命令，MCP 就退化成任意命令执行。
- Streamable HTTP 把风险移动到网络身份、服务发现和远端认证。

两种方式都需要治理，只是治理对象不同。

本节代码让 Host 使用当前 Python 解释器启动固定的 `server.py`，这是教学环境中的确定边界。迁移时，工程师应把 Server 配置放进受控注册表，而不是允许模型临时构造任意 URL 或命令。模型可以从已批准目录中选能力，不能自己扩张能力目录。

本阶段解决了“怎样发布清楚、可发现的订单能力”。你需要形成的能力是：设计 Tool 契约、选择原语、选择传输。但 Server 能列出工具，只证明能力存在；Host 还必须把远端声明转换成现有 Runtime 和 Loop 真正认识的对象——那是下一阶段的任务。

五、第三阶段——接入 Host：怎样让远端工具进入现有 Runtime？

5.1 为什么不能把 MCP Client 直接交给模型？

结论前置：模型只应看到工具声明并提出 Tool Call；真正的 `client.call_tool()` 必须被包装成 Runtime 的执行函数。

如果 Loop 收到工具名后直接调用 MCP Client，就会形成第二条执行通道——本地工具经过 Runtime，MCP 工具却绕过 Runtime。上一节建立的参数校验、取消、超时入口和错误归一化将全部失效。

因此，正确做法不是修改 Loop，而是引入一个 **MCP Adapter**：它把远端工具的名称、描述和输入 Schema 转成内部 `ToolDefinition`，再把 `client.call_tool()` 包装成这个工具的 handler。从 Runtime 看，它仍然只是在执行一个统一工具；从 Loop 看，它仍然只接收 `Tool Result`。工具究竟来自本地函数、stdio 子进程还是远程 HTTP，不应泄漏到这两层。

这就是 Harness 在本节中的位置：Harness 不替代 MCP 协议，而是规定 **MCP 能力以何种方式进入当前系统**。AI 可以生成 Adapter 的普通代码，但“所有工具必须经过同一个 Runtime”是工程师必须先确定的系统不变量。

5.2 多个 Server 的工具为什么必须使用限定名？

一个 Host 可能同时连接 `orders`、`github` 和 `filesystem`。它们都可能提供 `search`、`get` 或 `create`。如果只把原始工具名交给模型，后接入的工具可能覆盖前面的工具，审计记录也无法指出调用来自哪个 Server。

因此，Host 聚合工具时使用：

代码块

```
1 server_id.tool_name
```

订单工具变成 `orders.get_order`，GitHub 工具可以是 `github.search_issues`。限定名是 Host 的内部治理标识，不要求远端 Server 改名。

关键适配代码如下：

mcp_client_v1.py

代码块

```
1 async def discover_tools(client: Client, server_id: str) -> dict[str,
    ToolDefinition]:
2     result = await client.list_tools()
3     definitions = {}
4
5     for remote in result.tools:
6         qualified_name = f"{server_id}.{remote.name}"
7
8         async def handler(call_id, arguments, remote_name=remote.name):
9             response = await client.call_tool(remote_name, arguments)
10            return {"tool_call_id": call_id, **result_payload(response)}
11
12        definitions[qualified_name] = ToolDefinition(
13            name=qualified_name,
14            description=remote.description or "",
15            input_schema=remote.input_schema,
16            handler=handler,
17        )
18
19    return definitions
```

这段代码解决的是"远端 MCP Tool 怎样变成本地 Runtime Tool"。四个设计点最关键：

#	设计点	说明
1	list_tools() 返回值	工具列表位于 .tools，是结果对象而非裸列表
2	模型看到限定名，远端调用使用原始名	展示与调用的名称解耦
3	remote_name=remote.name 默认参数	固定当前循环变量，避免闭包把所有 handler 指向最后一个工具
4	result_payload() 统一归一化	在结果进入 Loop 前统一成功、失败和大小上限

工程师必须决定：`server_id` 的稳定来源、工具快照何时刷新、描述是否可信、哪些字段可以送给模型。AI 可以生成适配器，但它不知道工具目录变化是否会影响正在执行的 Run。稳妥做法是：**一个 Run 使用一份工具快照**，新目录从下一个 Run 生效，避免模型第一轮看见的 Schema 在第二轮悄悄变化。

工具发现也不应机械地发生在每一轮模型调用之前。发现结果通常进入 Host 的目录缓存，再按当前任务和策略形成 **Run 级快照**。如果 Server 临时不可用，Host 需要区分"没有这个工具"和"目录暂时无法刷新"：

- **前者**是确定性配置结果；
- **后者**可能继续使用仍在有效期内的旧快照，也可能拒绝启动新 Run。

这个选择取决于工具风险等级，不能由 SDK 自动替你决定。

目录更新至少要验证三个不变量：

1. 限定名仍然唯一；
2. 输入 Schema 可被当前 Runtime 支持；
3. 已开始的 Run 不被无提示修改。

对于删除或改名的工具，Host 还要考虑历史消息中是否仍有待关联的 Tool Call。**发现不是一个 UI 下拉列表问题，而是一次运行所依赖的能力契约。**

5.3 Client 的生命周期应该放在哪里？

本节锁定的 `mcp==2.0.0` 中，stdio 场景需要将 `StdioServerParameters` 交给 `stdio_client()`，再把 transport 交给 `Client`：

代码块

```
1 parameters = StdioServerParameters(  
2     command=sys.executable,  
3     args=[str(base_dir / "server.py")],  
4     cwd=base_dir,
```

```

5 )
6
7 async with Client(stdio_client(parameters)) as client:
8     tools = await discover_tools(client, "orders")
9     result = await client.call_tool("get_order", {"order_id": "ord_1001"})

```

进入 `async with` 才真正启动子进程并建立连接；离开代码块时 SDK 负责关闭连接和子进程。后续 SDK 文档可能提供 `Client(parameters)` 简写，但**不能把新文档示例与锁定的 2.0.0 混用**。本节以实际安装和运行结果为准；升级 SDK 时再通过 Adapter 契约测试确认调用方式。

课堂示例只有一个 Client，所以生命周期放在 `main()` 中。生产 Harness 应把它提升到 **Connection Manager**：按 `tenant` 与 `server_id` 定位连接、维护状态、处理健康检查和重连。这里先划边界，不在 MCP 基础章节展开连接池平台。

Connection Manager 对上层最好只暴露类似 `get_client(tenant, server_id)` 的稳定接口。它内部再处理状态转换：`disconnected` → `connecting` → `ready` → `error` → `reconnecting`。**Agent Loop 不应知道当前使用 stdio 还是 HTTP，也不应自己实现指数退避**。这样以后替换传输方式，只改变连接层，不改变 Tool Definition 和消息协议。

多租户场景下，Client 也不是越复用越好。连接中可能绑定租户凭证、服务端会话或缓存上下文。工程师需要根据 Server 身份模型决定连接是否允许共享。这个结论留到生产章节展开，但本节必须先避免把 Client 做成**无条件的全局单例**。

5.4 MCP 结果为什么不能原样回灌模型？

远端 Server 可能返回文本，也可能返回 `structuredContent`，还可能标记 `is_error`。结果可能很大，内容也不能因为“来自工具”就自动视为可信。

本节先做两个最小边界：

代码块

```

1 def result_payload(result: Any, max_chars: int = 2_000) -> dict[str, Any]:
2     if result.is_error:
3         text = getattr(result.content[0], "text", "MCP_TOOL_ERROR")
4         return {"ok": False, "code": "MCP_TOOL_ERROR", "message":
5             text[:max_chars]}
6     value = result.structured_content

```

```

7     if value is None:
8         value = {"text": getattr(result.content[0], "text", "")}
9
10    if len(json.dumps(value, ensure_ascii=False)) > max_chars:
11        return {"ok": False, "code": "RESULT_TOO_LARGE"}
12    return dict(value)

```

它解决两件事：

1. 把 MCP Tool Error 转成 Loop 可观察的结构化失败；
2. 阻止无限结果挤占上下文。

真正的生产实现还要做内容分类、脱敏和间接 Prompt 注入隔离——那属于后续工具安全章节。本节需要建立的判断是：协议调用成功不等于业务成功，拿到结果不等于可以原样交给模型。

结果归一化还要保留 `tool_call_id`。同一轮可能并行出现多个 Tool Call，Loop 必须把每份结果送回正确的调用。只凭工具名关联是不够的，因为模型可以在同一轮两次调用 `get_order` 查询不同订单。调用 ID 是运行时相关性，订单 ID 是业务标识，二者不能互相代替。

示例优先使用 `structured_content`，只有缺失时才退回文本。这不是为了追求形式统一，而是为了减少模型再次解析自然语言带来的歧义。但结构化也不等于可信：字段可能缺失、值可能过大，远端 Server 甚至可能是恶意的。下一节会在同一个 `result_payload()` 之后增加脱敏、内容标记与审计边界。

本阶段解决了远端工具进入 Runtime、名称聚合、Client 生命周期和结果归一化。你需要形成的能力是：写 Adapter 而不是改 Loop。调用链已经闭合，最后还要验证：成功、拒绝、失败和恢复是否真的走了不同路径——那是下一阶段的任务。

六、第四阶段——回到 Agent Loop：结果怎样驱动下一轮？

6.1 为什么接通一次 `call_tool()` 还不算 Agent？

结论是：单次协议调用只是 RPC；Tool Result 回到上下文并触发下一轮判断，才进入 Agent Loop。

最小 Loop 如下：

loop_v1.py

代码块

```

1  for round_no in range(1, max_rounds + 1):

```

```
2     reply = await provider.complete(messages, model_tools)
3
4     if reply.text is not None:
5         return reply.text
6
7     tool_result = await runtime.execute(reply.tool_call, tools)
8     messages.append({
9         "role": "tool",
10        "tool_call_id": reply.tool_call.id,
11        "content": tool_result,
12    })
13
14    raise RuntimeError("MAX_ROUNDS_EXCEEDED")
```

这段代码的重点不在循环语法，而在三个终止条件：

终止条件	含义	处理方式
reply.text is not None	模型给出最终文本	正常结束，返回结果
模型响应既无文本也无工具调用	协议级异常	以协议错误结束
超过 max_rounds	Loop 失控风险	Harness 强制停止，抛出异常

不能让模型自己保证"我不会无限调用"——最大轮次是 Harness 的安全网，不是模型的自觉。

失败返回后也不能一律由 Runtime 重试：

- 参数缺失 → Runtime 拒绝，不发起网络调用；
- 订单不存在 → 业务失败，交给模型改计划可能有用；
- 网络瞬断 → 才可能进入有限重试。

是否安全重试与业务幂等性有关，这个事实不能由模型猜测。

把示例中的四条路径再连起来看：

路径	行为	关键点
成功	模型提出 orders.get_order，Runtime 校验，Server 返回结构化订单，下一轮给最终答案	正常闭环
拒绝	调用缺少 order_id，Runtime 根据 Schema 在网络调用前返回 INVALID_ARGUMENT	远端未执行
失败	参数合法但订单不存在，Server 返回 Tool Error，Host 归一化为 MCP_TOOL_ERROR	远端已执行，业务未找到
恢复	Loop 看到失败结果后改查 ord_1002，得到结果后结束	模型改变计划，非盲目重试

这里的"恢复"不是 Runtime 自动把错误吃掉。Runtime 只负责把事实准确交回，Loop 根据任务决定新的动作。若错误来自 HTTP 503，Connection Manager 可能在安全边界内有限重试；若 `create_ticket` 已经可能成功但响应丢失，就必须**先用幂等键或查询状态确认**，不能再次盲目创建。

因此，**测试不能只断言最终一句话**。至少还要记录：Server 是否真的收到调用、调用次数、参数和顺序。参数拒绝场景如果最终显示 `INVALID_ARGUMENT`，但 Server 实际已经执行过，仍然是不合格实现。Agent 测试要**验证副作用与执行轨迹**，而不只是文本答案。

6.2 工程师与 AI 在这条主链中怎样分工？

工作	AI 可以帮助	工程师必须负责
Server 契约	根据既定接口生成装饰器与类型注解	工具颗粒度、读写边界、错误语义
Host Adapter	生成 SDK 调用与对象转换	限定名、快照、生命周期、兼容版本
Runtime	补校验、结果转换和测试样板	执行顺序、可信边界、超时与取消策略
Agent Loop	补消息转换和循环骨架	最大轮次、终止条件、失败由谁处理
验收	生成测试用例草稿	选择真实风险路径并检查副作用

这就是编码工作交给 AI 以后，工程师职责发生的变化。过去我们可能把主要时间花在 SDK 调用和数据类上；现在这些代码可以让 AI 快速生成，但我们要把更多时间**前移到工具规格和边界设计**，再把更多时间**后置到测试、Trace、评测和运行结果**。考核标准从"是否把 Client 写出来"变成"**是否有证据证明能力接入正确**"。

每完成一个阶段，都要问三个问题：

- 1. 这一步由 AI 生成了什么？
- 2. 哪些业务事实仍然由人决定？
- 3. 下一步用什么测试或 Trace 证明它成立？

如果回答不了第三个问题，就不要急着让 AI 继续生成下一层代码。

迁移到文件系统 MCP 时，不是把 `order_id` 改成 `path` 就结束。工程师还要重新决定：允许的工作目录、符号链接策略、结果大小上限和写操作边界。迁移到数据库时，要决定：只读账号、表范围、行数上限和查询超时。迁移到 GitHub 时，要决定：仓库范围、Token 权限和哪些写操作需要确认。**协议适配可以复用，业务事实必须重新判断。**

四类常见接入可以用同一套主链，但 Server 侧的业务边界不同：

接入对象	合适的 Tool 示例	不应直接暴露的能力
文件系统	<code>read_project_file</code> 、 <code>search_project</code>	任意路径读写、任意 Shell
数据库	<code>get_order</code> 、 <code>query_sales_summary</code>	带生产写权限的通用 SQL
GitHub	<code>get_pull_request</code> 、 <code>create_issue</code>	无仓库范围限制的长期个人 Token
内部 HTTP	<code>create_support_ticket</code>	原样透传 <code>method</code> 、 <code>URL</code> 、 <code>headers</code> 、 <code>body</code>

这张表不是本节要实现四套 Server，而是训练**工具颗粒度判断**。MCP 统一的是发现和调用方式，不会自动让一个危险的通用接口变安全。工程师越早把内部 API 收敛成任务级能力，后续权限、审批和审计越容易写成明确规则。

本阶段完成了**发现、选择、执行、观察与继续决策**的闭环。现在我们用一份完整代码证明这条链，而不是停留在架构图上——那是下一节的内容。

七、完整可运行演示：订单查询 MCP Server + Host

7.1 环境与运行方式

本示例锁定：

- Python 3.12

- `mcp==2.0.0`
- `jsonschema==4.26.0`
- `openai==1.109.1` （通过 OpenAI-compatible 接口调用 DeepSeek）

创建目录后保存下面两个文件，然后执行：

代码块

```
1 python3.12 -m venv .venv
2 source .venv/bin/activate
3 python -m pip install "mcp==2.0.0" "jsonschema==4.26.0" "openai==1.109.1"
4 python host.py
```

第一次不配置模型密钥，先运行 **MCP 契约测试**。需要验收真实 Agent Loop 时，再设置：

代码块

```
1 export DEEPSEEK_API_KEY="你的密钥"
2 export LLM_BASE_URL="https://api.deepseek.com"
3 export LLM_MODEL="deepseek-chat"
4 python host.py
```

如果阶段一网关已提供 OpenAI-compatible 地址，只替换 `LLM_BASE_URL`、`LLM_MODEL` 和对应密钥读取方式，**不修改 MCP Adapter、Runtime 与 Loop**。

`host.py` 会通过 stdio 自动启动 `server.py`。如需单独验证 Streamable HTTP：

代码块

```
1 python server.py --http
```

服务会监听 `http://127.0.0.1:8000`。课堂示例只验证传输启动，不把本地无鉴权地址扩展成公网服务。

7.2 `server.py`

代码块

```
1 from __future__ import annotations
2
3 import sys
4
5 from mcp.server import MCPServer
6 from mcp.server.mcpserver.exceptions import ToolError
```

```
7
8
9 mcp = MCPServer(
10     "order-service",
11     instructions="查询订单必须先调用 get_order，不要猜测订单状态。",
12 )
13
14 ORDERS = {
15     "ord_1001": {"order_id": "ord_1001", "status": "shipped", "amount": 199},
16     "ord_1002": {"order_id": "ord_1002", "status": "pending", "amount": 88},
17 }
18
19
20 @mcp.tool(structured_output=True)
21 def get_order(order_id: str) -> dict[str, object]:
22     """按订单号查询订单状态。找不到订单时返回工具错误。"""
23     order = ORDERS.get(order_id)
24     if not order:
25         raise ToolError("ORDER_NOT_FOUND")
26     return {"ok": True, **order}
27
28
29 @mcp.tool(structured_output=True)
30 def create_ticket(order_id: str, reason: str) -> dict[str, object]:
31     """为已存在的订单创建售后工单。"""
32     if order_id not in ORDERS:
33         raise ToolError("ORDER_NOT_FOUND")
34     if not 4 <= len(reason) <= 100:
35         raise ToolError("INVALID_REASON")
36     return {"ok": True, "ticket_id": "ticket_2001", "order_id": order_id}
37
38
39 @mcp.resource("policy://order-status")
40 def order_status_policy() -> str:
41     """订单状态说明，只提供只读上下文。"""
42     return "pending=待处理; shipped=已发货; cancelled=已取消"
43
44
45 @mcp.prompt()
46 def order_assistant(order_id: str) -> str:
47     """生成订单查询任务模板。"""
48     return f"查询订单 {order_id}。必须使用工具获取真实状态，不要猜测。"
49
50
51 if __name__ == "__main__":
52     if "--http" in sys.argv:
53         mcp.run("streamable-http", host="127.0.0.1", port=8000)
```

```
54     else:
55         mcp.run("stdio")
```

重点:

- `get_order()` 的**错误语义**——`ORDER_NOT_FOUND` 是稳定失败码, 非网络异常;
- **三种原语** (Tool / Resource / Prompt) 的职责边界;
- 末尾的**传输选择**——stdio 与 HTTP 共用同一套业务声明。

课堂上不逐行讲装饰器。真正要学会的是: 一个现有业务能力如何被压缩成稳定契约, 而不是怎样把所有内部 API 原样暴露出去。

7.3 `host.py`

代码块

```
1  from __future__ import annotations
2
3  import asyncio
4  import json
5  import os
6  import sys
7  from dataclasses import dataclass
8  from pathlib import Path
9  from typing import Any, Awaitable, Callable, Protocol
10
11 from jsonschema import ValidationError, validate
12 from mcp import Client, StdioServerParameters, stdio_client
13 from openai import AsyncOpenAI
14
15
16 @dataclass(frozen=True)
17 class ToolCall:
18     id: str
19     name: str
20     arguments: dict[str, Any]
21
22
23 @dataclass(frozen=True)
24 class ModelReply:
25     text: str | None = None
26     tool_call: ToolCall | None = None
27
28
29 class LLMPProvider(Protocol):
30     async def complete(
```

```

31         self,
32         messages: List[dict[str, Any]],
33         tools: List[dict[str, Any]],
34     ) -> ModelReply: ...
35
36
37 class DeepSeekProvider:
38     """复用 OpenAI-compatible 接口; 生产项目可替换为阶段一网关。"""
39
40     def __init__(self) -> None:
41         self.model = os.getenv("LLM_MODEL", "deepseek-chat")
42         self.client = AsyncOpenAI(
43             api_key=os.getenv("DEEPSEEK_API_KEY"),
44             base_url=os.getenv("LLM_BASE_URL", "https://api.deepseek.com"),
45         )
46
47     async def complete(
48         self,
49         messages: List[dict[str, Any]],
50         tools: List[dict[str, Any]],
51     ) -> ModelReply:
52         response = await self.client.chat.completions.create(
53             model=self.model,
54             messages=messages,
55             tools=tools,
56             tool_choice="auto",
57         )
58         message = response.choices[0].message
59         if message.tool_calls:
60             call = message.tool_calls[0]
61             return ModelReply(
62                 tool_call=ToolCall(
63                     id=call.id,
64                     name=call.function.name,
65                     arguments=json.loads(call.function.arguments or "{}"),
66                 )
67             )
68         return ModelReply(text=message.content or "")
69
70
71 ToolHandler = Callable[[str, dict[str, Any]], Awaitable[dict[str, Any]]]
72
73
74 @dataclass(frozen=True)
75 class ToolDefinition:
76     name: str
77     description: str

```

```

78     input_schema: dict[str, Any]
79     handler: ToolHandler
80
81     def to_model_tool(self) -> dict[str, Any]:
82         return {
83             "type": "function",
84             "function": {
85                 "name": self.name,
86                 "description": self.description,
87                 "parameters": self.input_schema,
88             },
89         }
90
91
92     class ToolRuntime:
93         async def execute(self, call: ToolCall, tools: dict[str, ToolDefinition]) -
94         > dict[str, Any]:
95             tool = tools.get(call.name)
96             if not tool:
97                 return {"ok": False, "code": "TOOL_NOT_FOUND"}
98             try:
99                 validate(instance=call.arguments, schema=tool.input_schema)
100             except ValidationError as exc:
101                 return {"ok": False, "code": "INVALID_ARGUMENT", "message":
102                 exc.message}
103             return await tool.handler(call.id, call.arguments)
104
105
106     def result_payload(result: Any, max_chars: int = 2_000) -> dict[str, Any]:
107         if result.is_error:
108             text = getattr(result.content[0], "text", "MCP_TOOL_ERROR")
109             return {"ok": False, "code": "MCP_TOOL_ERROR", "message":
110             text[:max_chars]}
111         value = result.structured_content
112         if value is None:
113             text = getattr(result.content[0], "text", "")
114             value = {"text": text}
115         encoded = json.dumps(value, ensure_ascii=False)
116         if len(encoded) > max_chars:
117             return {"ok": False, "code": "RESULT_TOO_LARGE"}
118         return dict(value)
119
120     async def discover_tools(client: Client, server_id: str) -> dict[str,
121     ToolDefinition]:
122         result = await client.list_tools()
123         definitions: dict[str, ToolDefinition] = {}

```



```

168         "type": "function",
169         "function": {
170             "name": reply.tool_call.name,
171             "arguments": json.dumps(
172                 reply.tool_call.arguments, ensure_ascii=False
173             ),
174         },
175     }
176 ],
177     }
178 )
179 tool_result = await runtime.execute(reply.tool_call, tools)
180 messages.append(
181     {
182         "role": "tool",
183         "tool_call_id": reply.tool_call.id,
184         "content": json.dumps(tool_result, ensure_ascii=False),
185     }
186 )
187
188 raise RuntimeError("MAX_ROUNDS_EXCEEDED")
189
190
191 async def main() -> None:
192     base_dir = Path(__file__).resolve().parent
193     parameters = StdioServerParameters(
194         command=sys.executable,
195         args=[str(base_dir / "server.py")],
196         cwd=base_dir,
197     )
198
199     async with Client(stdio_client(parameters)) as client:
200         tools = await discover_tools(client, "orders")
201         resources = await client.list_resources()
202         prompts = await client.list_prompts()
203
204         print("PROTOCOL", client.protocol_version)
205         print("TOOLS", sorted(tools))
206         print("RESOURCES", [str(item.uri) for item in resources.resources])
207         print("PROMPTS", [item.name for item in prompts.prompts])
208
209         runtime = ToolRuntime()
210         success = await runtime.execute(
211             ToolCall("call_success", "orders.get_order", {"order_id":
212 "ord_1001"}),
213             tools,

```

```

214     assert success["status"] == "shipped"
215     print("CONTRACT SUCCESS", success["status"])
216
217     rejected = await runtime.execute(
218         ToolCall("call_reject", "orders.get_order", {}), tools
219     )
220     assert rejected["code"] == "INVALID_ARGUMENT"
221     print("REJECT", rejected["code"])
222
223     failed = await runtime.execute(
224         ToolCall(
225             "call_fail", "orders.get_order", {"order_id": "ord_missing"}
226         ),
227         tools,
228     )
229     assert failed["code"] == "MCP_TOOL_ERROR"
230     print("FAIL", failed["code"])
231
232     if "DEEPSEEK_API_KEY" not in os.environ:
233         print("LIVE SKIP: 设置 DEEPSEEK_API_KEY 后运行真实 Agent Loop")
234         return
235
236     answer = await run_agent(
237         provider=DeepSeekProvider(),
238         runtime=runtime,
239         tools=tools,
240         user_input=(
241             "先查询订单 ord_missing; 如果工具明确返回订单不存在, "
242             "再查询 ord_1002, 并告诉我最终查到的订单状态。"
243         ),
244     )
245     print("LIVE", answer)
246
247
248 if __name__ == "__main__":
249     asyncio.run(main())

```

设计要点:

- 没有用 Fake Provider。无密钥时只运行确定性 MCP 契约测试; 有密钥时才进入真实模型驱动的 Agent Loop。
- 生产项目可以把 `DeepSeekProvider` 替换成第一章的 OpenAI-compatible Gateway, `discover_tools()`、`ToolRuntime` 与 `run_agent()` 的边界保持不变。

阅读时只抓四条路径:

组件	职责
discover_tools()	远端工具 → 内部 ToolDefinition（含限定名）
ToolRuntime.execute()	远端调用前的参数校验
result_payload()	结果进入 Loop 前的归一化与截断
run_agent()	决定继续（工具调用）还是停止（最终文本或超时）

它们共同说明：MCP 是接入层，不是整个 Harness。

7.4 实际运行结果

不配置模型密钥时，契约测试的预期输出为：

代码块

```
1  PROTOCOL 2026-07-28
2  TOOLS ['orders.create_ticket', 'orders.get_order']
3  RESOURCES ['policy://order-status']
4  PROMPTS ['order_assistant']
5  CONTRACT SUCCESS shipped
6  REJECT INVALID_ARGUMENT
7  FAIL MCP_TOOL_ERROR
8  LIVE SKIP: 设置 DEEPSEEK_API_KEY 后运行真实 Agent Loop
```

这份输出证明了三件确定性事实：

#	验证点	结论
1	Host 能通过 stdio 发现并调用远端 Tool	发现与调用链路打通
2	缺少 order_id 在 Runtime 被拒绝	远端未执行，拒绝在边界内完成
3	参数合法但订单不存在时返回稳定 Tool Error	业务失败与网络故障分离

配置 DeepSeek 后，继续验收**第四件事**：真实模型能否先调用 `orders.get_order` 查询不存在的订单，观察失败结果后改查 `ord_1002`，最后正常结束。

这里不把预录回复或 Fake Provider 的结果冒充模型验收。真实模型运行会受到模型版本影响，因此课程验收既要保存最终回答，也要保存 Tool Call 的名称、参数、顺序和 Tool Result。

Streamable HTTP 启动也已单独验证，Server 能在 `127.0.0.1:8000` 完成启动与关闭。这里不伪造真实模型结果，也不把“通过语法检查”冒充端到端运行。

7.5 怎样把课堂演示迁移成真实项目？

迁移时不要一次替换全部组件。建议保留这份确定性契约测试，然后分三步替换：

第一步：替换 Model Provider

只替换 `DeepSeekProvider` 为第一章已完成的 Model Adapter，检查真实模型是否能根据 `orders.get_order` 的名称、描述和 Schema 产生正确 Tool Call。此时 Server、Runtime 和失败样例均不变。若模型选择错误，先检查工具描述和消息转换，不要急着修改 Server 业务逻辑。

第二步：替换数据源

只替换 `ORDERS`，让 `get_order()` 调用真实订单仓储或内部 HTTP API，但保持工具名称、输入 Schema、结构化输出和错误码不变。Host 测试无需重写。真实下游的超时和认证信息由 Server 适配层处理，不能出现在模型参数里。

第三步：替换传输方式

把 stdio 换成 Streamable HTTP。Host Adapter 面向同一个 Client 能力，Tool Definition 和 Loop 不应随之改变。如果一次 transport 替换迫使你修改模型提示词、Runtime 校验和业务消息，说明抽象层已经泄漏。

每一步都要回归四条路径：

路径	验证要点
成功	正常返回结构化结果
拒绝	没有触发远端计数器
失败	保留稳定错误码
恢复	未超过轮次上限

这样排查时可以明确是 Model Adapter、MCP Adapter、Server 业务还是 Transport 出了问题，而不是面对一个“Agent 不工作”的模糊结论。

课堂上读代码也按这个顺序，不逐行朗读：

1. Server 契约
2. `discover_tools()` 的转换
3. `ToolRuntime` 的执行前边界
4. 结果怎样回到 Loop

能沿这四个检查点解释一次完整运行，比记住所有导入路径更重要。

八、MCP SDK 与 Harness 的边界

8.1 哪些已经有，哪些仍然要自己写？

能力	MCP 规范 / Python SDK	Host / Harness / 业务系统
Tools / Resources / Prompts	定义协议并提供客户端、服务端 API	选择原语、能力颗粒度和业务契约
stdio / Streamable HTTP	提供 transport	决定配置、部署、凭证与生命周期策略
工具参数 Schema	Server 声明，Client 可发现	转成内部 Tool，并补业务校验
Tool Call 执行循环	不提供完整业务 Agent Loop	把 MCP handler 注册进 Runtime，再交给 Loop
限定名与工具快照	不替 Host 决定	设计 <code>server_id.tool_name</code> 与 Run 级快照
权限、审批、审计	不自动提供完整治理平台	后续工具治理实现策略、确认与记录
多租户、连接池、重连	提供底层连接原语	Connection Manager 管理租户和连接状态
结果可信边界	提供结果结构	限长、脱敏、错误归一化和注入隔离

这张表要帮助大家避免两个极端：一是重复实现 SDK 已经处理的协议细节；二是误以为用了 SDK 就自动具备生产 Harness。工程师的价值在中间这层：把协议能力接到稳定的运行链上，并把业务正确性放在确定性代码里。

资料参考也要遵守同一原则。我们使用官方 Python SDK 的 `MCPServer` 和 `Client`，是为了让协议细节交给 SDK；教学代码中的 `ToolDefinition`、限定名、`Runtime`、`Loop` 和契约测试，则是 Host 为完成工程目标补充的 Harness 能力。不能因为 SDK 能列出和调用工具，就声称系统已经具备权限、审计、重试与生产连接管理。

当 AI 为你生成 MCP 代码时，审查顺序也应该固定：

1. 核对安装版本和官方导入路径；
2. 核对 Client 生命周期和真实返回对象；

- 3. 检查工具是否进入统一 Runtime；
- 4. 运行成功、拒绝、失败和恢复测试。

只看代码风格或类型检查，无法发现调用绕过边界、结果关联错误和重复副作用。

8.2 后续学习路线怎样安排？

附件路线图的八个阶段仍然保留，但按课程顺序重新分配篇幅：

阶段	本节处理程度	后续位置
1. 协议地基	讲清当前发现、版本兼容与三原语	本节完成
2. MCP Server	详细实现 stdio 与 Streamable HTTP	本节完成
3. MCP Host	详细实现发现、适配与 Loop	本节完成
4. 权限与审计	只标出 <code>Runtime.execute()</code> 前后插入点	下一节工具治理重点
5. 连接管理与多租户	只说明 Client 生命周期需上移	生产 Harness 专题
6. OpenTelemetry	只区分运维 Trace 与合规审计	可观测性专题
7. 安全加固	只声明 Tool Result 是不可信输入	工具安全专题
8. Elicitation 审批体验	了解 Server 可向用户索取输入	审批交互扩展

未来的生产目录可以逐步演进为：

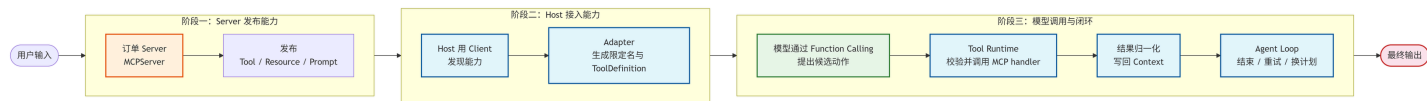
代码块

```
1  mcp_project/  
2  |— server/app.py  
3  |— host/llm_provider.py  
4  |— host/agent_loop.py  
5  |— host/tool_registry.py  
6  |— host/permission.py  
7  |— host/audit_store.py  
8  |— host/connection_manager.py  
9  |— host/tenant_context.py  
10 |— host/observability.py  
11 |— security/injection_guard.py
```

但本节只要求真正写完前三个核心职责：**Server、Host Adapter、Agent Loop 接入**。其余目录是路线图，不是要求大家在一个 MCP 基础章节里搭完平台。

九、本节总结：MCP 最终训练的是什么？

把全节收回一条主链：



MCP 只负责阶段一的发布和阶段二的发现/调用协议；阶段三的校验、归一化和决策闭环全部由 Host 的 Runtime 与 Loop 完成。三个阶段合在一起，才构成一个完整的 Agent 工具调用链路。

本节最重要的是掌握五个判断：

- 1. MCP 解决**标准化接入**，不替代 Function Calling、Runtime 和 Loop
- 2. Tool、Resource、Prompt 必须**按业务语义选择**，不为覆盖协议而滥用
- 3. 远端工具要**适配成 Harness 的统一 Tool 接口**，不能绕过 Runtime
- 4. 多 Server 聚合要使用**限定名**，MCP 结果进入 Context 前要**归一化和限长**
- 5. **协议成功不等于业务成功**，失败是否重试必须看失败类型与幂等性。

AI 可以帮助我们快速完成 Server 包装、SDK 调用和测试样板。Agent 开发工程师必须亲自决定的是：**工具边界、可信数据来源、错误语义、名称稳定性、连接生命周期、结果边界和 Loop 的终止条件**，并用测试和 Trace 证明这些判断真的生效。只有这些证据成立，AI 生成的代码才值得进入生产主链。

现在 MCP 工具已经能被统一接入了，但“能发现、能调用”仍然不能证明“当前用户有权调用、这次调用已经被审计、危险动作得到确认”——这正是下一节**工具治理**必然要解决的问题。

十、课堂练习

10.1 课堂练习

将订单案例扩展一个 `get_logistics(order_id)` Tool，要求：

- 1. 仍使用 `orders.get_logistics` 限定名；
- 2. 找不到物流时返回稳定业务错误，不抛出未处理异常；
- 3. Host 的 Runtime 不增加任何针对物流工具的特殊分支；
- 4. 结果超过上限时返回 `RESULT_TOO_LARGE`；

5. 模型先查询订单，只有 `status=shipped` 时才查询物流。

10.2 验收问题

1. 能否用一句话区分 Function Calling、MCP、Tool Runtime 与 Agent Loop?
2. 能否用测试区分参数拒绝、Tool Error、网络失败和 Loop 恢复?
3. 能否指出权限、审计与多租户为什么放在接入之后单独治理?

参考源码与资料

- MCP 规范: <https://modelcontextprotocol.io/specification/2026-07-28>
- Server discovery: <https://modelcontextprotocol.io/specification/2026-07-28/server/discover>
- MCP transports: <https://modelcontextprotocol.io/specification/2026-07-28/basic/transports>
- MCP Python SDK v2: <https://py.sdk.modelcontextprotocol.io/>
- Python SDK Client: <https://py.sdk.modelcontextprotocol.io/client/>